Laporan Capstone: Mini Distributed System

Bab 10, *Inside Modern AI Systems*. Proyek belajar, **tidak production-ready**. Data utama dari InstaBox `nanad` (dijalankan pemilik proyek), pembanding dari sandbox pengembangan. Verifikasi evidence dan tabel lengkap ada di `COMPARISON.md`.

1. Masalah dan user flow

Membuat laporan pesanan terlalu lama untuk satu request sinkron. Request bisa timeout, user mengklik ulang (kerja ganda), dan proses yang mati tidak meninggalkan jejak.

Flow: client `POST /jobs` (opsional `Idempotency-Key`) → `202` + `jobId` → worker mengerjakan → client membaca `GET /jobs/:id` sampai `SUCCEEDED`/`FAILED` → laporan di `REPORT_DIR/<jobId>.json`, dengan sha256 tercatat di database. Operator memakai `/metrics`, log JSON, dan `npm run check`.

2. Arsitektur dan eksperimen

```
client --HTTP--> API (1 proses) --SQL--> SQLite: jobs, job_events <--SQL-- worker (1..n proses) --> REPORT_DIR/*.json
```

Dependency: hanya Node.js >= 22.13 (`node:sqlite`, `node:http`, `node:test`). Tanpa paket npm, Docker, atau broker.

Mekanisme	Kegagalan yang ditangani
Claim dalam transaksi `BEGIN IMMEDIATE`	Dua worker mengambil job yang sama
Lease 5 s + heartbeat 1 s; lease kedaluwarsa diambil alih	Worker mati atau macet
Fencing: tulisan status wajib cocok (`lease_owner, attempts`)	Worker zombie menimpa hasil
Retry bounded (3 attempt, backoff eksponensial + jitter, hanya error aman diulang)	Gangguan sementara tanpa retry storm
Timeout per attempt	Job hang memegang worker selamanya
File atomik dulu, lalu commit `SUCCEEDED` + sha256	Crash di antara file dan database
Idempotency-Key, backpressure 429, timeout HTTP 408/413	Request ganda, banjir, client lambat

Iterasi 1-4 mengikuti brief: (1) baseline, (2) fault tolerance, (3) visibility (event per attempt, error class, metrik antrean), (4) review arsitektur (ADR-001). Setelah itu, audit menemukan dan memperbaiki 1 P0 (satu URL malformed mematikan API), 2 P1, dan 11 P2 (`AUDIT.md`). Perbaikan yang bisa direproduksi punya test yang gagal di kode lama.

Metodologi

- **Kode:** ZIP final (commit `abb9371`). SHA-256 ZIP yang diuji di `nanad` cocok dengan ZIP hasil build. Markdown evidence identik byte per byte dengan render ulang dari JSON. Data JSON lulus pemeriksaan konsistensi internal.
- **Environment:**

Environment	Node	CPU	RAM	Kernel
nanad	22.23.3	4 vCPU AMD EPYC	8 GiB	6.18.54-instabox
Sandbox	22.22.0	4 vCPU Xeon	16 GiB	6.18.44

Test 58/58 dilaporkan lulus di `nanad` (log tidak dilampirkan). Di sandbox, test diulang 10x berturut-turut tanpa gagal.

- **Ablation:**
  - 200 job (seed 42): 127 normal, 65 gangguan sementara, 4 rusak permanen, 4 hang.
  - 3 worker. `kill -9` pada worker 1 di t=0.8 s dan worker 2 di t=1.6 s, masing-masing diganti.
  - Batas tunggu 15 s; "macet" = belum final saat itu.
  - 5 konfigurasi x 3 repetisi. Angka: median (min-max), median = nilai tengah bawah.
- **Drain:** antrean diisi tanpa worker, lalu k worker dinyalakan bersamaan.
- **POST:** 1000 request dari satu client berurutan, yang mengukur batas client, bukan kapasitas API.
- **Storage:** semua eksperimen memakai database di `os.tmpdir()`.
- **Invariant:** `checkConsistency` dijalankan di akhir setiap run.

Hasil: reliability (`nanad`; sandbox dalam kurung siku)

Konfigurasi	Sukses	Gagal	Macet	Semua final
Baseline	125 (99-125) [99 (99-125)]	63 (50-63)	12 (12-51) [50 (12-51)]	tidak pernah
+ retry saja	125 (99-125) [128 (102-129)]	4	71 (71-97) [68 (67-94)]	tidak pernah
+ timeout saja	126 (125-127) [125 (125-126)]	72 (71-73)	2 [2]	tidak pernah

Konfigurasi	Sukses	Gagal	Macet	Semua final
+ lease recovery saja	124 (99-125) [100 (98-124)]	65 (52-66)	10 (10-49) [49 (10-49)]	tidak pernah
Semua	**192 [192]**	**8**	**0 [0]**	**10275 ms (9928-10324)** [10490]

- **Tidak ada satu mekanisme yang cukup sendirian.**
  - *Retry saja* membuat lebih banyak job macet (71-97): job yang di-retry kembali antre di belakang worker yang ditahan job hang.
  - *Timeout saja* membebaskan worker, tapi 2 job milik worker yang di-kill tetap `RUNNING`.
  - *Recovery saja* mengubah 2 job itu menjadi `FAILED WORKER_LOST` yang terlihat. Dengan `MAX_ATTEMPTS=1` job itu tidak dijalankan ulang. Laporan versi sebelumnya keliru menyebut job itu "diselamatkan".
- **Konfigurasi parsial bimodal di kedua environment.** Contoh: baseline macet 12/12/51 di `nanad` dan 51/50/12 di sandbox. Median berbeda hanya karena mode mana yang menang 2 dari 3 repetisi, jadi yang bermakna hanya rentangnya.
- **Konfigurasi lengkap sama di 6/6 repetisi (dua environment):** 192 sukses, 8 gagal yang memang seharusnya gagal (4 `PERMANENT` tanpa retry, 4 `TIMEOUT` setelah 3 attempt), 0 macet.
- **Invariant: 0 violation dan 0 sukses ganda di 68 run.**

Trade-off latency

p50 / p95 end-to-end job sukses:

	nanad	**Sandbox**
Baseline	2077 / 5843 ms	2009 / 5693 ms
Konfigurasi lengkap	3741 / 7854 ms	3633 / 7989 ms

Di `nanad`, p50 naik sekitar 1.7 s. Penyebabnya:

1. 86-88 retry kembali ke antrean yang sama.
2. Setiap job hang memakan 3 x 1 s waktu worker sebelum `TIMEOUT`.
3. Baseline tampak lebih cepat karena bias survivorship: latency hanya dihitung dari job yang lolos. Job yang gagal cepat atau macet tidak ikut.

Latency ini didominasi delay simulasi, backoff, dan timeout, sehingga hampir sama di kedua environment (selisih p50 sekitar 3%).

Hasil: concurrency dan storage

Ukuran (median)	**`nanad` 1 / 2 / 4 worker**	**Sandbox 1 / 2 / 4 worker**
Drain job ringan (job/s)	4717 / 4184 / 3571 (1.00 / 0.89 / 0.76x)	406 / 455 / 522 (1.00 / 1.12 / 1.29x)
Drain job menunggu I/O 20 ms (job/s)	48 / 96 / 192 (4.00x)	42 / 81 / 162 (3.86x)
POST p50 / p95 / p99, 1 client	0.27 / 1.42 / 3.64 ms	1.88 / 10.5 / 21.62 ms

- **Job yang menunggu I/O naik linear di kedua environment.**
- **Job ringan tidak naik linear di keduanya**, konsisten dengan satu penulis SQLite: setiap claim dan commit antre di satu kunci tulis. Besarnya penurunan di `nanad` (0.76x) belum bisa diatribusikan sepenuhnya ke kontensi. Drain hanya 0.2-0.3 s, sehingga booting 3 proses Node tambahan ikut terukur.
- **`nanad` 6-12x lebih cepat untuk operasi yang berat di storage** (drain job ringan 6.8-11.6x, POST 5.9-7.4x), padahal hampir sama untuk workload yang didominasi I/O simulasi. *Hipotesis, belum diverifikasi:* fsync di `/tmp` milik `nanad` jauh lebih murah (misalnya tmpfs). Kalau benar, throughput itu tidak mewakili database di disk persisten. Angka throughput tidak boleh dipindahkan antar mesin tanpa diukur ulang.

3. Keputusan

- **Tetap sederhana:** satu codebase, satu API, worker terpisah (n instance), satu file SQLite. Tanpa broker, DB server, container, atau orchestrator. Data dari dua environment tidak menunjukkan bottleneck yang membenarkannya untuk beban capstone (ADR-001).
- **Yang dipisah:** worker dari API, demi isolasi crash dan CPU (teruji: worker mati atau dibekukan tidak mengganggu API).
- **Menambah worker** hanya untuk job yang mahal per-claim (I/O atau CPU), bukan untuk job mikro.
- **Kompromi yang disadari:**
  - At-least-once, bukan exactly-once. Aman karena laporan byte-identik antar attempt dan tulisan status di-fence.
  - File yatim bisa tertinggal untuk job `FAILED` (terdeteksi `npm run check`).
- **Keterbatasan:**
  - Tanpa fsync file laporan (risiko listrik padam belum terbukti).
  - Tanpa autentikasi (API dikunci ke loopback). Satu host, satu penulis.
  - Lock SQLite memblokir event loop maksimal `BUSY_TIMEOUT_MS`.
  - Test bergantung timing.

    - Eksperimen mengukur storage `/tmp`, bukan `data/`.
- **Langkah berikutnya:**
    1. Cek `df -T /tmp "$(pwd)/data"` di `nanad` untuk menguji hipotesis fsync.
    2. Drain dengan N lebih besar untuk memisahkan kontensi dari ongkos startup.
    3. fsync bila `report_missing` pernah muncul.
    4. Retensi data dan alert antrean.
    5. Outbox sebelum ada side effect eksternal.
    6. PostgreSQL hanya bila trigger ADR terpenuhi.